Strings I: Hashing, Prefix Function, Z-Function

Linear-time pattern matching and O(1) substring comparison

Learning goals

- Know why naive matching is O(n·m) and when that hurts
- Compute **polynomial hashes** of all substrings in O(1) after O(n) preprocessing
- Avoid **collisions** and **anti-hash tests** (random base, mod 2^61 − 1)
- Compute the **prefix function** and run **KMP** in O(n + m)
- Compute the **Z-function** in O(n)
- Solve borders, periods, counting occurrences, and string compression

Motivating problem — CSES 1753 *String Matching*

Text t and pattern p, |t|, |p| ≤ 10^6. Count occurrences of p in t.

```
t = saippuakauppias
p = pp              → 2 occurrences
```

Naive: compare p at each position — O(|t|·|p|) worst case:

```
t = aaaaaaaaaaaaaaaa…a      p = aaaa…ab
```

10^12 operations. We need O(|t| + |p|).

Part 1 — Polynomial hashing

Map a string to a number:

```
H(s) = s[0]·B^(k−1) + s[1]·B^(k−2) + … + s[k−1]   (mod M)
```

- Equal strings → equal hashes (always)
- Different strings → equal hashes with probability ≈ k / M (random B)
- Compare hashes in O(1) instead of strings in O(k)

Prefix hashes → any substring in O(1)

```
h[0] = 0,   h[i+1] = h[i]·B + s[i]          (mod M)
pw[0] = 1,  pw[i+1] = pw[i]·B
hash(s[l..r)) = h[r] − h[l]·pw[r − l]        (mod M)
```

Like prefix sums (Week 7), but "shift then subtract".

```
s = a b c d,  hash(s[1..3)) = h[3] − h[1]·B² = (aB²+bB+c) − aB² = bB + c ✔
```

Choosing the modulus

Choice	Risk
mod 2^64 (overflow)	**broken** by Thue–Morse strings for any base
fixed base, mod 10^9+7	birthday collisions at ~10^5 strings; anti-hash tests
two mods (10^9+7, 10^9+9), random bases	OK, ~2× slower
mod 2^61 − 1, random base	collision ≈ n / 2^61 per comparison ✔

Always pick the base **at random at runtime** on Codeforces (hacks!).

mod 2^61 – 1 multiplication

```
typedef unsigned long long ull;
const ull MOD = (1ULL << 61) - 1;
ull mulmod(ull a, ull b) {
    __uint128_t c = (__uint128_t)a * b;
    ull r = (ull)(c & MOD) + (ull)(c >> 61);   // 2^61 ≡ 1 (mod MOD)
    return r >= MOD ? r - MOD : r;
}
```

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
ull base = rng() % (MOD - 1000) + 500;
```

What hashing buys you

- Substring equality in O(1) → many O(n^2) brute forces become fast
- **Periods** (CSES 1733): p is a period ⇔ `s[0..n−p) == s[p..n)` → O(n) total
- Longest common prefix of two suffixes: binary search on length, O(log n)
- Palindrome check: hash of s vs hash of reverse(s)
- Count distinct substrings of a given length: hash set

Cost: probabilistic — a collision gives **wrong answer**, silently.

Part 2 — Borders and the prefix function

A **border** of s is a proper prefix that is also a suffix.

```
s = abcababcab
borders: "ab", "abcab"   (lengths 2, 5)
```

Prefix function: `π[i]` = length of the longest border of `s[0..i]`.

```
s : a b c a b a b c a b
π : 0 0 0 1 2 1 2 3 4 5
```

Key lemma

The borders of `s[0..i]` are exactly:

```
π[i],  π[π[i] − 1],  π[π[π[i] − 1] − 1],  …   (until 0)
```

A border of a border is a border; the longest border shorter than k is `π[k − 1]`.

→ To extend, try the longest border, then fall back along this chain.

Computing π in O(n)

```
vector<int> prefixFunction(const string& s) {
    int n = s.size();
    vector<int> pi(n, 0);
    for (int i = 1; i < n; i++) {
        int k = pi[i - 1];
        while (k > 0 && s[i] != s[k]) k = pi[k - 1];
        if (s[i] == s[k]) k++;
        pi[i] = k;
    }
    return pi;
}
```

Amortized: k increases by ≤ 1 per step, each fallback decreases it → total O(n).

Trace on `abcababcab`

```
i=3 'a': k=0, s[0]='a' match → π=1
i=4 'b': k=1, s[1]='b' match → π=2
i=5 'a': k=2, s[2]='c' ≠ 'a' → k=π[1]=0; s[0]='a' match → π=1
i=6 'b': k=1, s[1]='b' match → π=2
i=7 'c': k=2, s[2]='c' match → π=3
i=8 'a': → π=4,  i=9 'b': → π=5
```

Borders of the whole string: 5, π[4] = 2, π[1] = 0 → {5, 2} ✔

KMP matching

Option 1: compute π of `p + '#' + t`; occurrences where `π[i] == |p|`.

Option 2: run the automaton over t without concatenating:

```
int k = 0, count = 0;
for (char c : t) {
    while (k > 0 && c != p[k]) k = pi[k - 1];
    if (c == p[k]) k++;
    if (k == m) { count++; k = pi[k - 1]; }
}
```

O(|t| + |p|), memory O(|p|), overlapping matches handled.

Periods from π

- Smallest period of s: `n − π[n − 1]`
- s is a power of a shorter string ⇔ `n % (n − π[n−1]) == 0` and `π[n−1] > 0`
- All periods: `n − b` for each border b (plus n)

```
s = abcabca, n = 7, π[6] = 4 → smallest period 3
periods: 7−4 = 3, 7−1 = 6, 7
```

Part 3 — Z-function

`z[i]` = length of the longest common prefix of `s` and `s[i..]` (z[0] := 0)

```
s : a a b x a a b
z : 0 1 0 0 3 1 0
```

- Pattern matching: Z of `p + '#' + t`, positions with `z[i] == |p|`
- Border of length k ⇔ `z[n − k] == k`
- Period p ⇔ `z[p] == n − p` (or p = n)

Computing Z in O(n): the [l, r) window

Maintain the match window `[l, r)` with the largest r: `s[l..r) == s[0..r-l)`.

```
vector<int> z(n, 0);
for (int i = 1, l = 0, r = 0; i < n; i++) {
    if (i < r) z[i] = min(r - i, z[i - l]);      // reuse mirror value
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] > r) { l = i; r = i + z[i]; }
}
```

Each successful comparison in the `while` pushes r right → O(n) total.

Why the mirror works

```
s:      [ s[0 .. r-l) ]
        0    i-l        r-l
s:           [ s[l .. r) ]
             l    i       r
```

Inside the window, `s[i..r)` equals `s[i-l..r-l)`, so z[i] ≥ min(r – i, z[i – l]). Only beyond r do we need fresh comparisons.

Prefix function vs Z-function

	Prefix function	Z-function
Definition	longest border ending at i	LCP with s starting at i
Streaming over text	✔ (automaton)	needs concatenation
Borders / periods	via chain	direct test
Interconvertible	✔ in O(n)	✔ in O(n)

Pick whichever makes the problem's condition easier to state.

Choosing a tool

Task	Best tool
Count/find occurrences	KMP or Z
Many substring comparisons (arbitrary positions)	hashing
Borders, periods, compression	prefix function / Z
LCP of two arbitrary suffixes	hashing + binary search (or suffix array, Week 29)
Many patterns at once	Aho–Corasick (Week 29)

Common pitfalls

- Hashing with mod 2^64 or a fixed base → hacked
- Forgetting `+ MOD` before taking the difference (unsigned underflow)
- `s[i]` as `char` may be negative → cast to `unsigned char` (and avoid value 0)
- π: off-by-one — fallback is `k = pi[k - 1]`, not `pi[k]`
- KMP: after a full match, continue from `pi[m - 1]` (or overlaps are missed)
- Separator `#` must not occur in either string
- Z-function: `z[0]` is a convention — don't treat it as n by accident

Practice problems

- CSES 1753 *String Matching*
- CSES 1732 *Finding Borders* · CSES 1733 *Finding Periods*
- CSES 1110 *Minimal Rotation* · CSES 1111 *Longest Palindrome* (hashing + binary search)
- Codeforces 126B *Password*
- Codeforces 432D *Prefixes and Suffixes*
- Codeforces 1200E *Compress Words*
- Codeforces 471D *MUH and Cube Walls*
- Codeforces 271D *Good Substrings* (hashing)

Summary

- **Hashing**: prefix hashes, O(1) substring compare; random base, mod 2^61 − 1
- **Prefix function**: longest border; fallback chain; O(n) amortized
- **KMP**: automaton over text, O(n + m)
- **Z-function**: LCP with prefix; [l, r) window, O(n)
- Borders and periods: `n − π[n−1]`, `z[n−k] == k`, `z[p] == n − p`
- Next week: tries, Aho–Corasick, suffix arrays